

Python for Web Development

Lesson 6: User Authentication and Authorization

User Authentication and Authorization - Study Notes

Key Concepts

- **Authentication vs. Authorization** – Authentication verifies *who* a user is; authorization determines *what* they are allowed to do.
- **Password Hashing & Salting** – Transforming plaintext passwords into irreversible hashes (e.g., bcrypt, Argon2) with a unique salt to thwart rainbow table attacks.
- **Session Management & Tokens** – Maintaining state via server side sessions, signed cookies, or stateless JWTs (JSON Web Tokens) and handling token expiration/revocation.
- **Role Based Access Control (RBAC)** – Assigning permissions to roles (e.g., admin, editor, viewer) and checking a user's role before granting access to resources.
- **Secure Credential Storage & Transmission** – Using HTTPS, protecting secrets (API keys, DB credentials) in environment variables or secret managers, and enforcing strong password policies.

Summary

User authentication is the process of confirming a user's identity, typically through credentials such as a username/email and password. Once identity is established, authorization decides which actions the authenticated user may perform within the application. Modern systems avoid storing plaintext passwords; instead, they hash passwords with a strong, adaptive algorithm (bcrypt, scrypt, Argon2) and a unique salt per user, making brute force and rainbow table attacks infeasible.

To keep users logged in without re-entering credentials on every request, applications employ session mechanisms—either server side session stores (e.g., Redis) with a session ID cookie, or stateless tokens like JWTs that carry claims and are signed with a secret key. Proper token handling includes setting short lifespans, refreshing tokens securely, and revoking compromised tokens.

Authorization is commonly implemented via Role Based Access Control (RBAC) or Attribute Based Access Control (ABAC). After authentication, the system looks up the user's role(s) or attributes and evaluates them against policy rules (e.g., "only admins can delete users"). Combining robust authentication with fine grained authorization ensures that only legitimate users can access the appropriate resources, protecting both data integrity and user privacy.

Important Points

- **Never store plaintext passwords**; always hash with a salt using a slow, memory hard function (bcrypt, scrypt, Argon2).
- **Use HTTPS everywhere** to protect credentials in transit; enforce HSTS.
- **Session cookies** should be `HttpOnly`, `Secure`, and `SameSite=Strict` (or `Lax`) to mitigate XSS and CSRF.
- **JWTs** are stateless but cannot be revoked without a blacklist or short expiry; consider refresh token rotation.
- **Implement rate limiting** on login/registration endpoints to deter credential stuffing and brute force attacks.
- **Validate and sanitize** all input (email, username) to prevent injection and overflow attacks.

- **Log authentication events** (success/failure, IP, timestamp) for anomaly detection and forensic analysis.
- **Follow the principle of least privilege**: assign users the minimal role/permissions needed for their tasks.
- **Regularly update dependencies** (e.g., password hashing libraries, auth frameworks) to patch known vulnerabilities.
- **Test authentication flows** with both positive and negative cases (invalid credentials, missing tokens, role escalation attempts).

Examples

Example 1: User Registration with bcrypt (Node.js/Express)

```
````javascript
const express = require('express');
const bcrypt = require('bcrypt');
const User = require('./models/User'); // Sequelize/Mongoose model
const router = express.Router();

router.post('/register', async (req, res) => {
 const { email, password } = req.body;
 // Basic validation
 if (!email || !password) {
 return res.status(400).json({ error: 'Email and password required' });
 }
 try {
 // Generate a salt and hash the password (cost factor 12)
 const salt = await bcrypt.genSalt(12);
 const hash = await bcrypt.hash(password, salt);
 // Store user
 const user = await User.create({ email, passwordHash: hash });
 res.status(201).json({ id: user.id, email: user.email });
 } catch (err) {
 // Handle duplicate email, etc.
 res.status(400).json({ error: err.message });
 }
});
module.exports = router;
````
```

Explanation:

- The password is never stored plaintext; `bcrypt.genSalt` creates a unique salt, and `bcrypt.hash` produces a salt embedded hash.
- The cost factor (12) makes the hash computationally expensive, slowing brute force attempts.
- Errors such as duplicate email are caught and returned with a 400 status.

Example 2: Role Based Middleware Check (Express + JWT)

```

```javascript
const jwt = require('jsonwebtoken');
const SECRET = process.env.JWT_SECRET;
// Middleware to verify JWT
function authenticateToken(req, res, next) {
 const authHeader = req.headers['authorization'];
 const token = authHeader && authHeader.split(' ')[1]; // Bearer <token>
 if (!token) return res.sendStatus(401);
 jwt.verify(token, SECRET, (err, user) => {
 if (err) return res.sendStatus(403); // invalid or expired token
 req.user = user; // { id, email, role }
 next();
 });
}
// Middleware to enforce role
function authorizeRole(...allowedRoles) {
 return (req, res, next) => {
 if (!req.user) return res.sendStatus(401);
 if (!allowedRoles.includes(req.user.role)) {
 return res.sendStatus(403); // insufficient privileges
 }
 next();
 };
}
// Usage
router.get('/admin/dashboard',
 authenticateToken,
 authorizeRole('admin'), // only users with role 'admin' can access
 (req, res) => {
 res.json({ message: 'Welcome to admin dashboard' });
 }
);
```

```

****Explanation:****

- `authenticateToken` validates the JWT signature and extracts the payload (including `role`).
- `authorizeRole` is a higher order middleware that checks whether the user's role is in the list of allowed roles; if not, it returns 403 Forbidden.
- This separation keeps authentication concerns distinct from authorization logic.

Quick Review

1. ****What is the primary difference between authentication and authorization?****
2. ****Why is it important to use a salt when hashing passwords?****

3. **Name two properties that secure session cookies should have.**
4. **How does a JWT differ from a traditional server side session?**
5. **In an RBAC system, what determines whether a user can perform a specific action?**

Further Reading

- **OWASP Authentication Cheat Sheet** – best practices for secure login mechanisms.
- **OWASP Authorization Cheat Sheet** – guidance on implementing RBAC, ABAC, and policy enforcement.
- **“Password Hashing Competition” (PHC) winners** – deep dive into Argon2, bcrypt, scrypt, and their parameters.
- **JSON Web Token (JWT) RFC 7519** – official specification and security considerations.
- **“Designing Secure Single Sign On (SSO) Systems”** – exploring OAuth 2.0, OpenID Connect, and federated identity.
- **“Breaking Down Session Management”** – articles on server side stores vs. stateless tokens, token revocation strategies, and refresh token rotation.
- **Security headers** – HSTS, CSP, Referrer Policy, and their role in protecting authentication flows.

These notes are intended as a study guide for an intermediate level lesson on user authentication and authorization. Adjust code snippets to match your specific stack and security requirements.

